

Guía de Instalación — RedsocialPeriferia

Esta guía está pensada para quien recibe el proyecto por primera vez y necesita levantarlo desde cero. Se cubren el método recomendado con Docker Compose, el método alternativo para desarrollo local sin Docker y la resolución de los problemas más frecuentes.

1. Prerequisitos

Método recomendado (Docker Compose)

Solo se requieren dos herramientas instaladas en el sistema:

Docker Desktop

- Descargar desde: <https://www.docker.com/products/docker-desktop/>
- Versión mínima requerida: Docker Desktop 4.x
- Durante la instalación, habilitar la integración con **WSL2** cuando el asistente lo solicite (es el backend recomendado para Windows).
- Después de instalar, verificar que Docker Desktop esté corriendo (ícono en la barra del sistema).
- Verificar la instalación abriendo una terminal y ejecutando:

```
docker --version
docker compose version
```

Deben mostrarse las versiones instaladas sin errores.

Git

- Descargar desde: <https://git-scm.com/downloads>
- Verificar la instalación:

```
git --version
```

Método alternativo (desarrollo local sin Docker para los servicios)

Si se desea ejecutar los microservicios fuera de Docker para facilitar el desarrollo, se necesitan adicionalmente:

Node.js 20+ (para el frontend Angular)

- Descargar desde: <https://nodejs.org/> (versión LTS recomendada)
- Verificar:

```
node --version    # debe mostrar v20.x.x o superior
npm --version
```

Java 21 y Maven (para el backend)

- Descargar JDK 21 desde: <https://adoptium.net/> (Eclipse Temurin 21)
- Maven se puede obtener desde: <https://maven.apache.org/download.cgi>
- O instalar mediante `winget` :

```
winget install EclipseAdoptium.Temurin.21.JDK
winget install Apache.Maven
```

- Verificar:

```
java -version    # debe mostrar openjdk version "21"
mvn -version
```

2. Clonar el repositorio

```
git clone https://github.com/usuario/redsocalperiferia.git
cd redsocialperiferia
```

Reemplazar la URL con la URL real del repositorio.

3. Levantar con Docker Compose (método recomendado)

Desde la raíz del repositorio, ejecutar:

```
docker-compose up --build
```

Este único comando realiza automáticamente las siguientes acciones:

1. **Descarga las imágenes base** desde Docker Hub si no están en caché local:
 - `postgres:16-alpine` — base de datos
 - `eclipse-temurin:21-jdk-alpine` — compilación de los microservicios Java
 - `eclipse-temurin:21-jre-alpine` — ejecución de los microservicios Java
 - `node:20-alpine` — compilación del frontend Angular
 - `nginx:alpine` — servidor web del frontend
2. **Construye el microservicio auth-service** ejecutando `mvn package -DskipTests` dentro del contenedor de build.
3. **Construye el microservicio post-service** de la misma forma.
4. **Construye el frontend Angular** ejecutando `npm install` y `npm run build --configuration production`, luego copia el resultado a nginx.
5. **Inicializa la base de datos PostgreSQL** ejecutando automáticamente los scripts en orden:
 - `db/01_schema.sql` — crea las tablas `users`, `posts` y `likes` con sus índices
 - `db/02_procedures.sql` — crea los stored procedures `sp_create_post` y `sp_add_like`, y la función `fn_get_posts_with_likes`
 - `db/03_seed.sql` — inserta los 5 usuarios de prueba y una publicación por cada uno
6. **Levanta todos los contenedores** dentro de la red interna `rsp-network` (bridge):
 - `rsp-postgres` en el puerto 5432
 - `rsp-auth-service` en el puerto 8081 (espera a que postgres esté `healthy`)

- `rsp-post-service` en el puerto 8082 (espera a que postgres esté `healthy`)
- `rsp-frontend` en el puerto 4200 (espera a `auth-service` y `post-service`)

La primera ejecución tarda más tiempo porque Docker debe descargar las imágenes base y compilar los proyectos. Las ejecuciones posteriores son considerablemente más rápidas gracias a la caché de capas.

4. Verificar que todo está funcionando

Esperar el arranque completo

Los microservicios Java requieren entre **2 y 3 minutos** para iniciarse completamente la primera vez (Spring Boot inicia, establece conexión con la base de datos y despliega los endpoints). Es normal ver mensajes de reintento de conexión en los logs durante ese período.

Para ver los logs en tiempo real:

```
# Ver logs de todos los servicios
docker-compose logs -f

# Ver logs de un servicio específico
docker-compose logs -f auth-service
docker-compose logs -f post-service
docker-compose logs -f frontend
```

La señal de que los servicios Java están listos es la línea en los logs similar a:

```
Started AuthServiceApplication in X.XXX seconds
Started PostServiceApplication in X.XXX seconds
```

Verificaciones paso a paso

1. Frontend:

- Abrir <http://localhost:4200>
- Debe mostrar la pantalla de login de RedsocialPeriferia

2. Swagger de auth-service:

- Abrir <http://localhost:8081/docs>
- Debe mostrar la documentación interactiva de la API de autenticación

3. Swagger de post-service:

- Abrir <http://localhost:8082/docs>
- Debe mostrar la documentación interactiva de la API de publicaciones

4. Prueba de login:

- En <http://localhost:4200> ingresar:
 - Usuario: `admin`
 - Contraseña: `Admin123!`
- Debe redirigir al feed de publicaciones mostrando los posts de los demás usuarios

5. Prueba de like en tiempo real:

- Abrir la aplicación en dos pestañas o navegadores diferentes con usuarios distintos
- Dar like a una publicación desde una pestaña
- El contador en la otra pestaña debe actualizarse automáticamente sin recargar

5. Ejecución sin Docker (alternativa para desarrollo local)

Este método permite ejecutar los servicios directamente en la máquina local, lo que facilita el uso de un IDE para depuración y desarrollo activo.

Base de datos

Levantar únicamente el contenedor de PostgreSQL:

```
docker-compose up postgres -d
```

Esto iniciará la base de datos en el puerto 5432 con el usuario `rspadmin`, la base de datos `redsocalperiferia` y ejecutará automáticamente los tres scripts SQL de inicialización.

Para verificar la conexión se puede usar pgAdmin o la herramienta de línea de comandos `psql`:

```
# Con psql (si está instalado localmente)
psql -h localhost -p 5432 -U rspadmin -d redsocialperiferia
# Contraseña: rsp_secure_pass_2024
```

Backend — auth-service

```
cd redsocialperiferiaback/auth-service
mvn spring-boot:run
```

El servicio estará disponible en <http://localhost:8081>.

Backend — post-service

Abrir una segunda terminal:

```
cd redsocialperiferiaback/post-service
mvn spring-boot:run
```

El servicio estará disponible en <http://localhost:8082>.

Frontend

Abrir una tercera terminal:

```
cd redsocialperiferiafront
npm install
npm start
```

Angular CLI levantará el servidor de desarrollo en <http://localhost:4200> con recarga automática al modificar archivos.

Nota: En el modo de desarrollo local, el proxy de Angular CLI (`ng serve`) enruta las peticiones a los servicios Java directamente. En el modo Docker, es nginx quien actúa como proxy inverso.

6. Ejecutar pruebas unitarias

Backend — auth-service

```
cd redsocialperiferiaback/auth-service
mvn test
```

Ejecuta los tests unitarios de `AuthService` (5 casos de prueba). No requiere base de datos; todos los repositorios y dependencias están mockeados con Mockito.

Backend — post-service

```
cd redsocialperiferiaback/post-service
mvn test
```

Ejecuta los tests unitarios de `PostService` (7 casos de prueba). Incluye verificación del broadcast WebSocket a través de un mock de `SimpMessagingTemplate` .

Frontend

```
cd redsocialperiferiafront
npm test
```

Ejecuta los tests de `AuthStore` y `PostsStore` con Karma y Jasmine en modo headless. Cubre: inicialización de estado, carga de posts, creación de publicaciones, actualización de likes por REST y por WebSocket STOMP.

Nota para Windows: Los tests de frontend requieren Google Chrome instalado (usado por `karma-chrome-launcher`). Si Chrome no está disponible, se puede configurar `karma.conf.js` para usar `ChromeHeadless` .

7. Detener los servicios

Detener los contenedores conservando los datos

```
docker-compose down
```

Los datos almacenados en PostgreSQL se conservan en el volumen `postgres_data` . Al volver a ejecutar `docker-compose up` , la base de datos retomará el estado donde se dejó (no re-ejecuta los scripts SQL porque los datos ya existen).

Detener los contenedores y eliminar todos los datos

```
docker-compose down -v
```

La opción `-v` elimina el volumen `postgres_data`. La próxima vez que se ejecute `docker-compose up`, la base de datos se inicializará desde cero con los scripts SQL (esquema, procedimientos y datos seed).

Esto es útil para tener un entorno limpio para pruebas o cuando se realizaron cambios en los scripts SQL.

8. Solución de problemas frecuentes

Puerto ya en uso

Síntoma: Error al iniciar `docker-compose up` con el mensaje `Bind for 0.0.0.0:XXXX failed: port is already allocated`.

Solución: Identificar qué proceso está usando el puerto y detenerlo, o modificar los puertos en `docker-compose.yml`. Por ejemplo, para cambiar el frontend del puerto 4200 al 4300:

```
frontend:
  ports:
    - "4300:80"    # cambiar 4200 por el puerto disponible
```

Los puertos más frecuentemente en conflicto son:

- 5432 — PostgreSQL local ya instalado en el sistema
- 8081 / 8082 — otros servicios Java corriendo
- 4200 — otro servidor de Angular CLI activo

Los contenedores Java tardan en iniciar o se reinician

Síntoma: Los logs muestran mensajes como `Connection refused` o el contenedor se reinicia repetidamente.

Causa: Es comportamiento normal durante los primeros minutos. El healthcheck de PostgreSQL debe reportar `healthy` antes de que los servicios Java intenten conectarse. Docker Compose gestiona esta dependencia automáticamente con `depends_on: condition: service_healthy`.

Acción: Esperar de 2 a 3 minutos. Si después de ese tiempo el problema persiste, revisar los logs de postgres:

```
docker-compose logs postgres
```

El frontend muestra error 502 Bad Gateway

Síntoma: La pantalla de login carga pero al intentar iniciar sesión aparece error 502.

Causa: Los microservicios Java aún no han terminado de iniciar cuando se hace la primera petición.

Solución: Esperar hasta ver en los logs los mensajes `Started AuthServiceApplication` y `Started PostServiceApplication`. Luego recargar la página.

Error de CORS en el navegador

Síntoma: La consola del navegador muestra errores CORS al hacer peticiones a los servicios.

Causa probable: El frontend no está accediendo desde `http://localhost:4200`. Los servicios Java solo permiten origen `http://localhost:4200` (configurado en `WebSocketConfig` y `SecurityConfig`).

Solución: Asegurarse de acceder al frontend exactamente desde `http://localhost:4200` , no desde `http://127.0.0.1:4200` ni desde otra dirección o puerto.

La base de datos no tiene datos seed

Síntoma: Se puede acceder al login pero las credenciales de los usuarios de prueba no funcionan.

Causa probable: El volumen de datos ya existía de una ejecución anterior donde los scripts no se ejecutaron correctamente.

Solución: Eliminar el volumen y reiniciar:

```
docker-compose down -v
docker-compose up --build
```

Error al ejecutar tests de frontend: Chrome no encontrado

Síntoma: `npm test` falla con `No binary for ChromeHeadless browser on your platform` .

Solución: Instalar Google Chrome o modificar `karma.conf.js` para usar un launcher diferente disponible en el sistema.

9. Flujo de la aplicación

Esta sección describe en detalle cómo funcionan los procesos principales de la aplicación.

Autenticación y gestión del token

1. El usuario introduce su usuario y contraseña en la pantalla de login (`/login`).
2. El componente `LoginComponent` invoca el método `login()` del `AuthStore` .
3. El `AuthStore` llama al `AuthService` , que realiza `POST /api/auth/login` hacia el auth-service.
4. El auth-service verifica las credenciales contra PostgreSQL usando BCrypt y, si son correctas, genera un JWT firmado con `JWT_SECRET` con una expiración de 24 horas.
5. El JWT es recibido por el frontend y guardado en `localStorage` bajo la clave `rsp_token` .
6. El `AuthStore` actualiza su estado de señales con el token y los datos del usuario.
7. El router Angular navega automáticamente a `/posts` .

Protección de rutas

- El `authGuard` protege las rutas `/posts` , `/posts/new` y `/profile` .
- Al acceder a cualquiera de estas rutas, el guard verifica si existe un token válido en el `AuthStore` .
- Si no hay token, redirige a `/login` .
- Al recargar la página, el `AuthStore` inicializa su estado leyendo `rsp_token` desde `localStorage` , por lo que la sesión persiste entre recargas.

Peticiones autenticadas al backend

- El `authInterceptor` intercepta todas las peticiones HTTP salientes.
- Si el `AuthStore` tiene un token, añade automáticamente el header `Authorization: Bearer <token>` a cada petición.
- Esto aplica tanto a las llamadas al auth-service (`/api/auth/profile`) como al post-service (`/api/posts`).

- El nginx actúa como proxy inverso: enruta `/api/auth/*` al auth-service (puerto 8081) y `/api/posts/*` al post-service (puerto 8082), de modo que el frontend solo conoce su propio origen.

Feed de publicaciones

1. Al cargar la ruta `/posts`, el `PostsListComponent` invoca `PostsStore.loadPosts()`.
2. El `PostsStore` llama a `PostService.getPosts()`, que hace `GET /api/posts`.
3. El post-service extrae el `userId` del JWT, consulta PostgreSQL excluyendo los posts propios del usuario y retorna el feed ordenado del más reciente al más antiguo.
4. El componente renderiza cada publicación con el alias del autor, el mensaje, la fecha y el contador de likes.
5. Simultáneamente, el `PostsListComponent` conecta el `WebSocketService` y suscribe a `/topic/likes/{postId}` por cada publicación visible.

Creación de publicaciones

1. El usuario navega a `/posts/new` y escribe un mensaje (máximo 500 caracteres).
2. El `CreatePostComponent` invoca `PostsStore.createPost()`.
3. El `PostsStore` llama a `PostService.createPost()`, que hace `POST /api/posts` con `{ "message": "...", "userId": ... }`.
4. El post-service valida el mensaje y llama al stored procedure `sp_create_post` en PostgreSQL, que inserta el registro y retorna el `post_id` y `published_at` generados por la base de datos.
5. Tras la creación exitosa, el `PostsStore` navega automáticamente a `/posts`.

Likes en tiempo real

1. El usuario hace clic en el botón de like de una publicación.
2. El `PostsListComponent` invoca `PostsStore.likePost(postId)`.
3. El `PostsStore` llama a `PostService.likePost(postId)`, que hace `POST /api/posts/{postId}/likes`.
4. El post-service llama al stored procedure `sp_add_like` en PostgreSQL. Si el usuario ya dio like anteriormente, el stored procedure usa `ON CONFLICT DO NOTHING` y simplemente retorna el conteo actual (la operación es idempotente).
5. El post-service envía el nuevo conteo de likes al topic WebSocket `/topic/likes/{postId}` mediante `SimpMessagingTemplate.convertAndSend()`.
6. Todos los clientes conectados y suscritos a ese tópico (incluyendo el mismo usuario que dio el like y otros en distintos navegadores o pestañas) reciben el mensaje `{ "postId": X, "likesCount": N }`.
7. El `WebSocketService` emite la actualización al `Subject<LikeUpdate>`, que es consumido por el `PostsStore` a través de `store.updateLikeCount(update)`.
8. El `PostsStore` actualiza de forma inmutable el array de posts, reemplazando el `likesCount` del post afectado.
9. Angular detecta el cambio en la señal y actualiza el contador en la vista sin recargar la página.

Cierre de sesión

1. El usuario hace clic en el botón de cerrar sesión (disponible en el perfil o en la barra de navegación).
2. El `AuthStore.logout()` elimina `rsp_token` de `localStorage`.
3. Limpia el estado de señales (token, user, profile).
4. El router navega a `/login`.
5. El `WebSocketService` desconecta la sesión STOMP al destruirse el componente del feed.